Machine Learning

PHYS 453

Dr. Daugherity

Short Intro to LLMs

- Final(?) topic this semester
- Speedrun ideas in NLP
- Intro to transformers and attention mechanisms that have a huge variety of applications far beyond LLMs…
- … but mostly focus on concepts that make LLMs work

Short Intro to LLMs

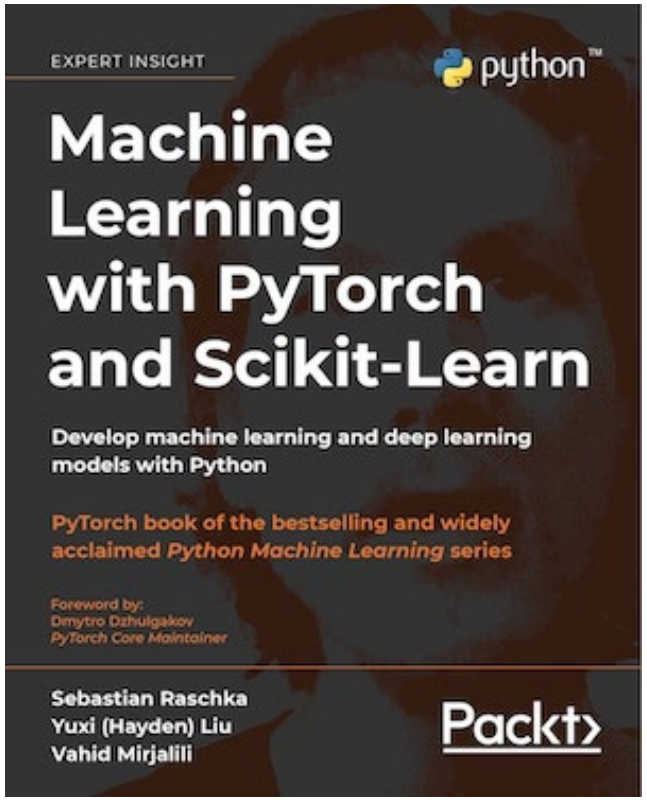
Want you to have an intuitive idea for why these tools are well-suited to these problems. Example: in image processing we needed tools sensitive to patterns in neighboring pixels, not absolute position in the image -> CNNs.

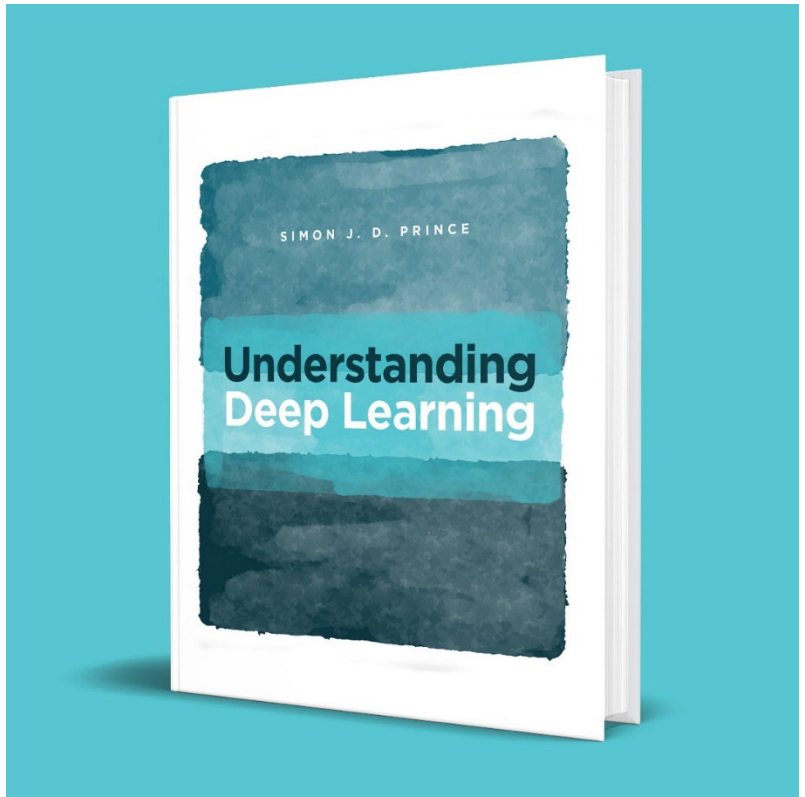
Challenges for LLMs:

- a hard problem: need flexible architecture that scales up to order of 10^{12} parameters
- parallelizable: if we require trillions of calculations per word, must be able to be processed in parallel or it will be too slow
- semantics: words have multiple meanings highly dependent on context

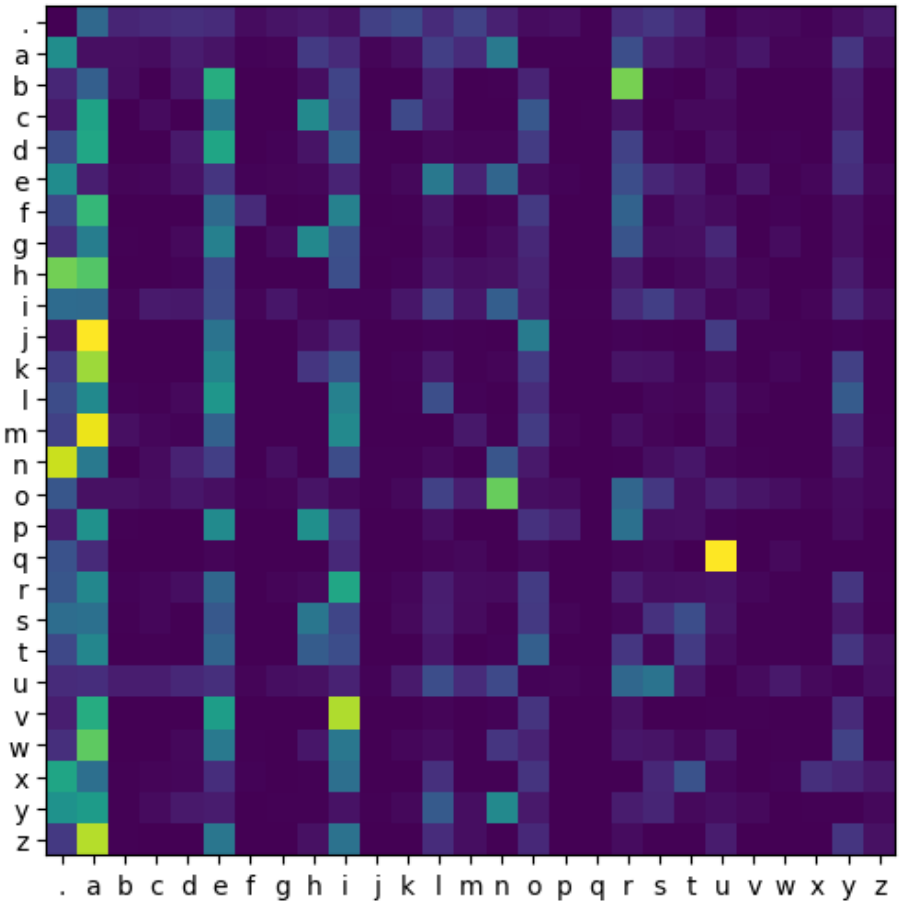
Sources

- https://github.com/rasbt/machine-learning-book/tree/main
- https://udlbook.github.io/udlbook/
- https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi 3Blue1Brown Neural Networks videos
- https://www.youtube.com/watch?v=VMj-3S1tku0&list=PLAqhIrjkxbuWI23v9cThsA9GvCAUhRvKZ&index=2 Andrej Karpathy's Neural Networks: Zero to Hero





N-Grams Aren't Very Good



- Bigram model: given one letter predict likely next letter
- Notice the q->u bright spot
- Trained on 32,000 names
- Doesn't work very well

Training data

```
1 words[:10]
```

```
['emma',
 'olivia',
 'ava',
 'isabella',
 'sophia',
 'charlotte',
 'mia',
 'amelia',
 'harper',
 'evelyn']
```

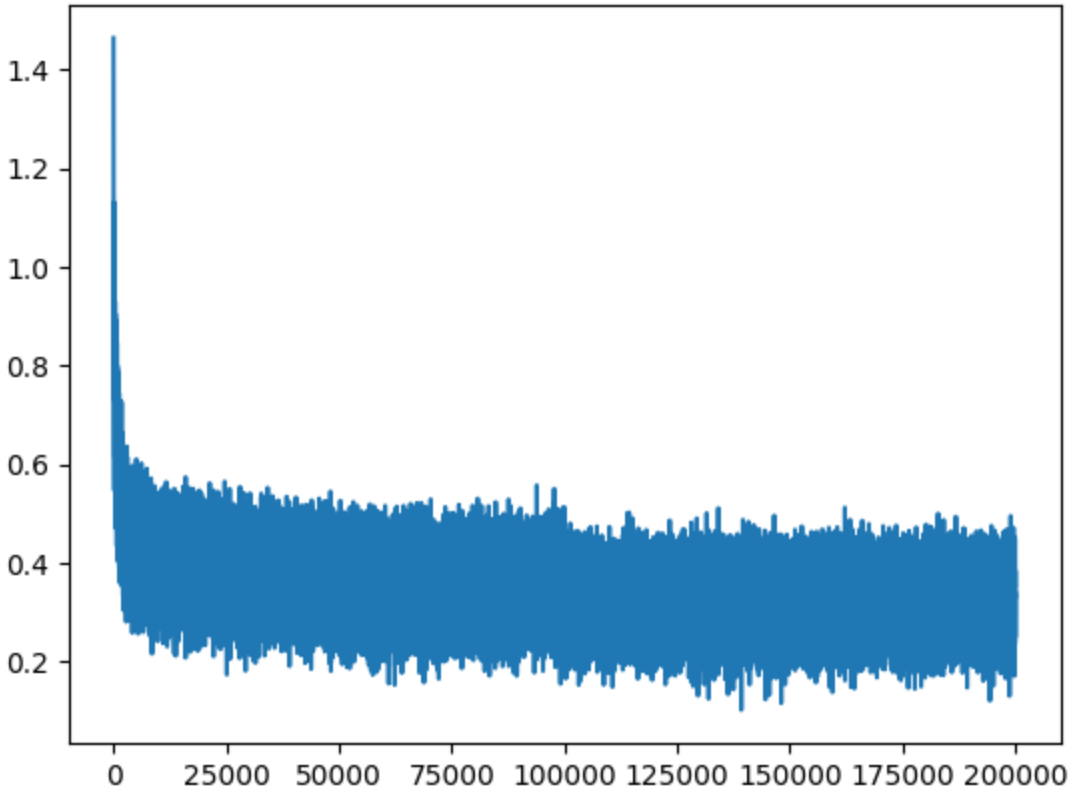
Generated Names

```
1 for i in range(10): print(make_name())
```

```
brosa
telyn
ka
saa
on
kallikanniche
as
delllamcira
deto
dier
```

N-Grams Aren’t Very Good

Using 3 input characters plus embedding (explained later)



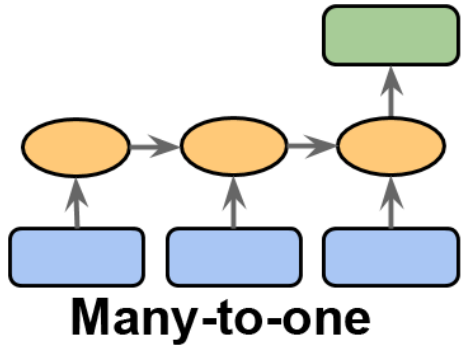

```
mora.
mayanniee.
med.
ryam.
rethan.
ejara.
graderedgeliigeli.
jen.
edelieananaraelyn.
malkia.
noshitergiaghiest.
jaio.
jenslen.
puor.
urzey.
der.
yarue.
els.
kayshianny.
mahia.
```

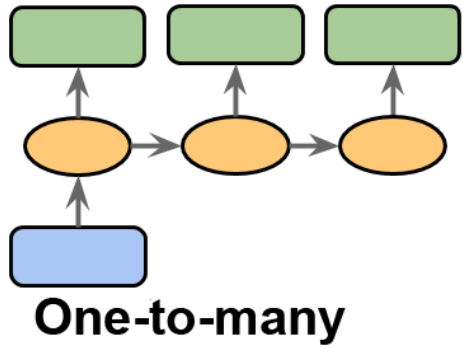
Working with sequences

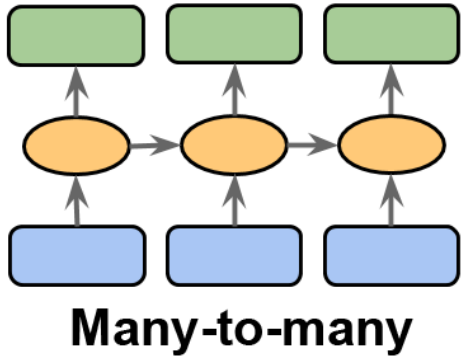
RECURRENT NN

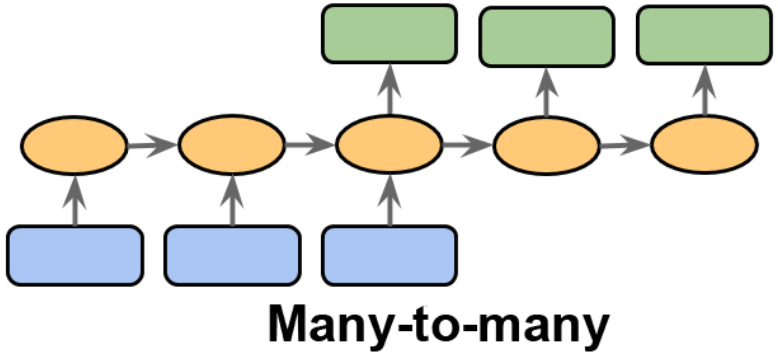
Sequences

- First problem: how do we deal with variable-length input?
- Different problem types to consider



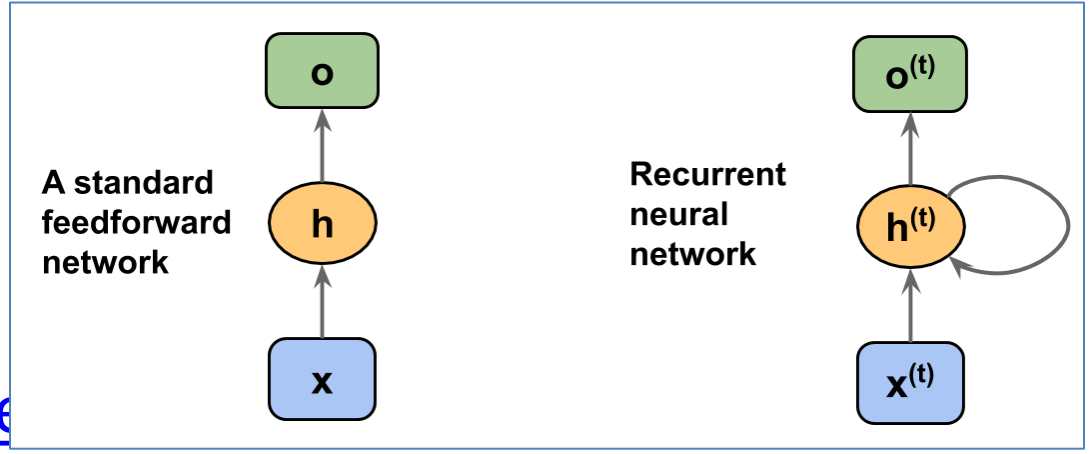


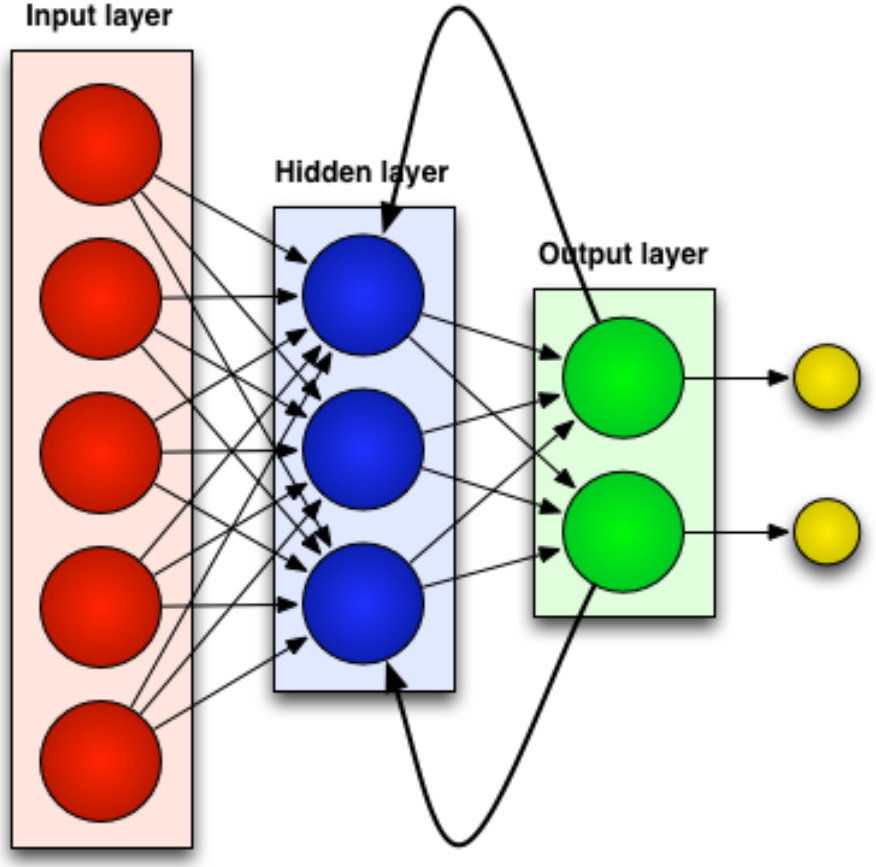




Recurrent Neural Network

- Adds limited “short term memory” to NN
- Allows for context in classifiers
- https://en.wikipedia.org/wiki/Recurrent_neural_network
- Can take previous steps as inputs





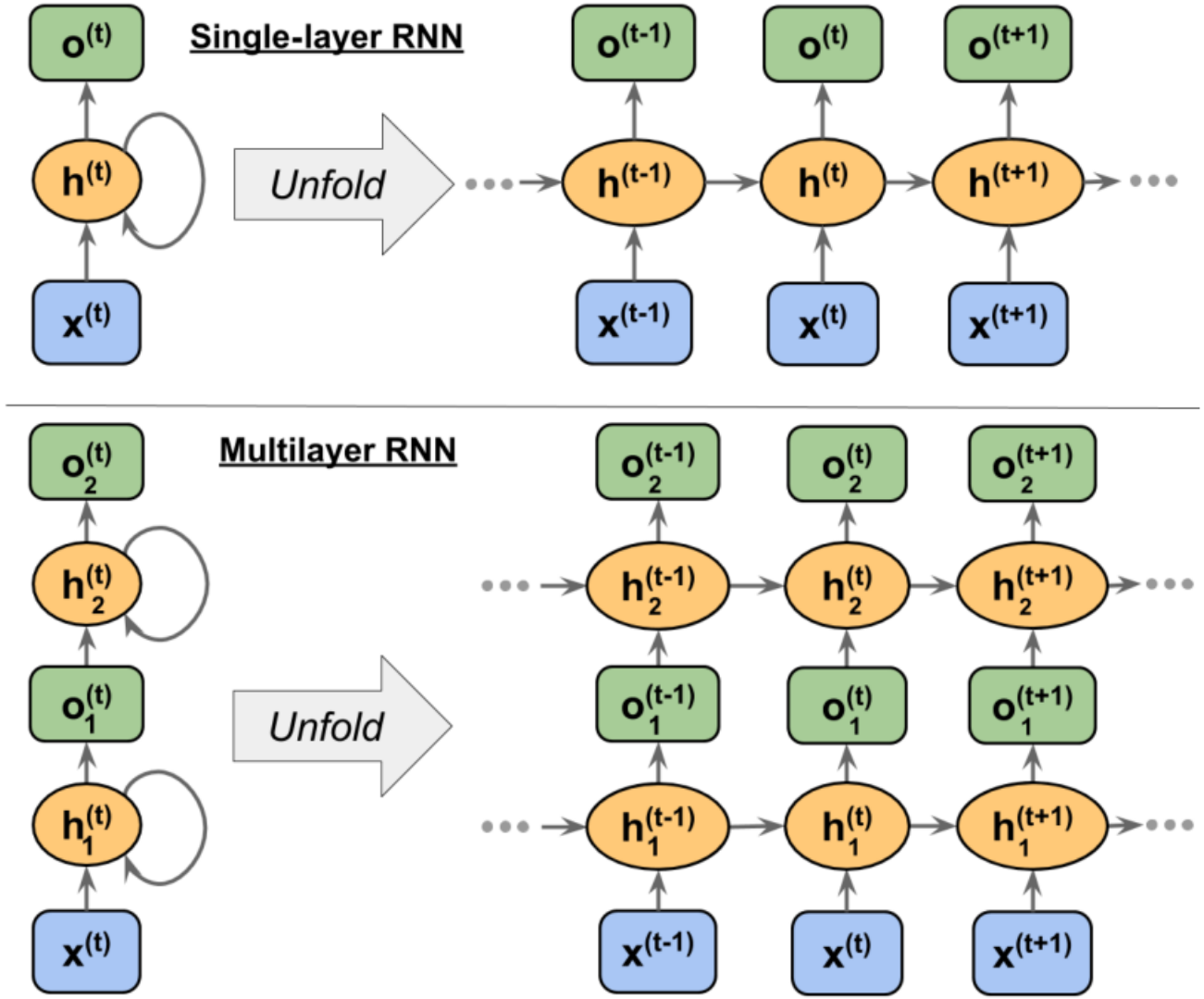


Figure 15.4: Examples of an RNN with one and two hidden layers

https://docs.pytorch.org/docs/stable/generated/torch.nn.RNN.html

class **torch.nn.RNN(*input_size, hidden_size, num_layers=1, nonlinearity='tanh', bias=True, batch_first=False, dropout=0.0, bidirectional=False, device=None, dtype=None*)** [source]

Apply a multi-layer Elman RNN with $\tanh$ or ReLU non-linearity to an input sequence. For each element in the input sequence, each layer computes the following function:

$$h_t = \tanh(x_t W_{ih}^T + b_{ih} + h_{t-1} W_{hh}^T + b_{hh})$$

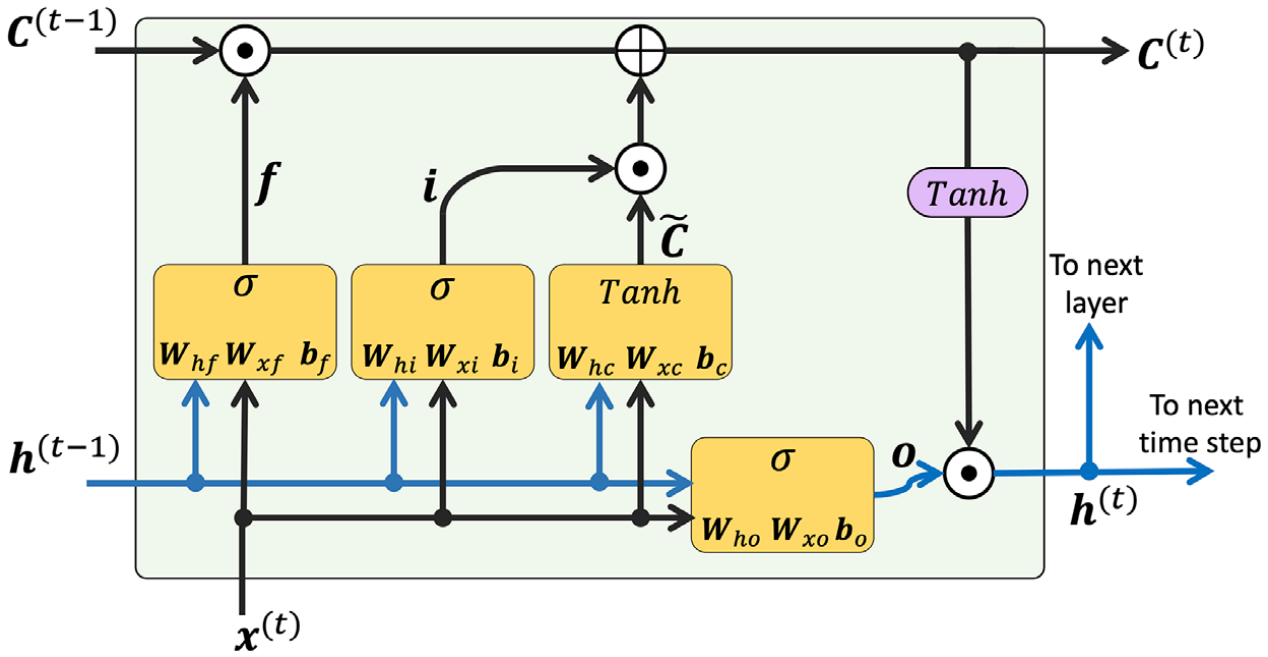
where h_t is the hidden state at time *t*, x_t is the input at time *t*, and $h_{(t-1)}$ is the hidden state of the previous layer at time *t-1* or the initial hidden state at time *0*. If `nonlinearity` is `'relu'`, then ReLU is used instead of $\tanh$.

See lots of cool examples of RNNs at:
https://karpathy.github.io/2015/05/21/rnn-effectiveness/

Upgrades to basic RNN

- One significant issue with basic RNNs is vanishing/exploding gradients. Inputs get multiplied by weights at hidden layer. If these values are much bigger or smaller than one, they quickly get out of control for large sequences
- The “memory” also tends to be very short
- Newer architectures solve these problems

Long short-term Memory (LSTM)



- f = forget gate, allows memory to reset
- C gets updated through gates i and $\tilde{C}$
- C has effect on output

Spoilers

New architectures did improve RNNs, but fundamental flaws remain with processing words in order like this:

- too hard to store all of connections of semantic meaning between all words, like trying to push too much data through a single pipe
- not parallel enough: since we take inputs from previous words, we can process them all in parallel and are forced to process them in order individually

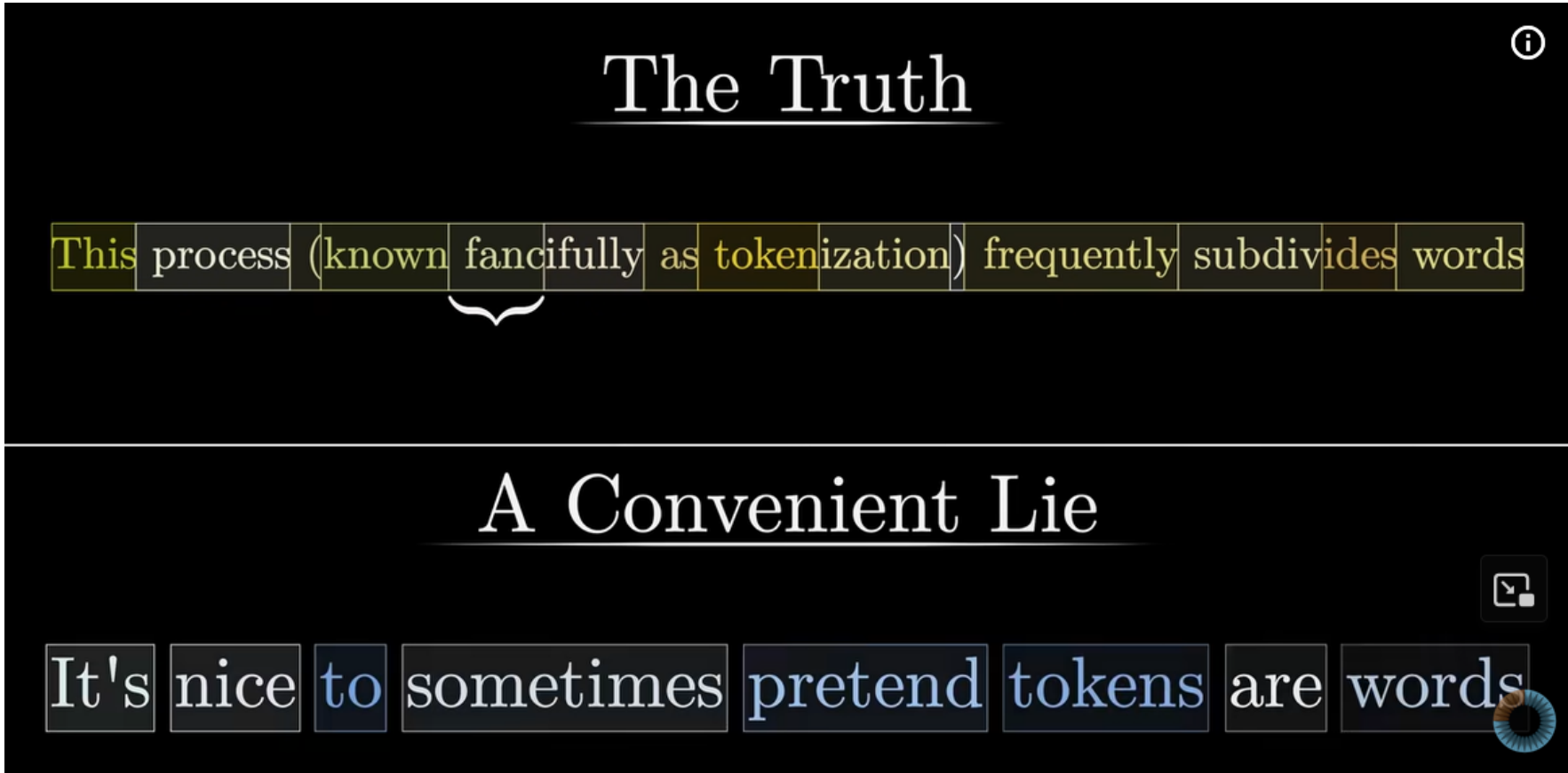
These problems get solved by transformers and attention made famous in 2016 doing translation. Remember, RNNs are useful and have many great applications, but not in state-of-the-art LLMs

TOKENS AND EMBEDDING

Tokens

Make dictionary of words, word fragments, and characters

```
>>> print([vocab[token] for token in ['this', 'is',
...     'an', 'example']])
[11, 7, 35, 457]
```



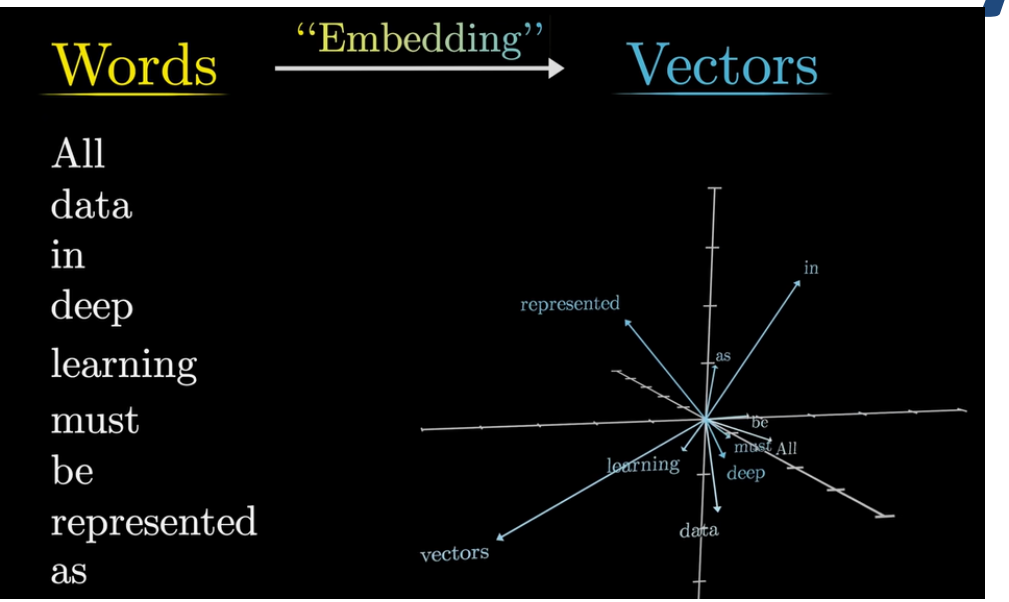
Why Embedding?

The straightforward approach doesn't work very well:

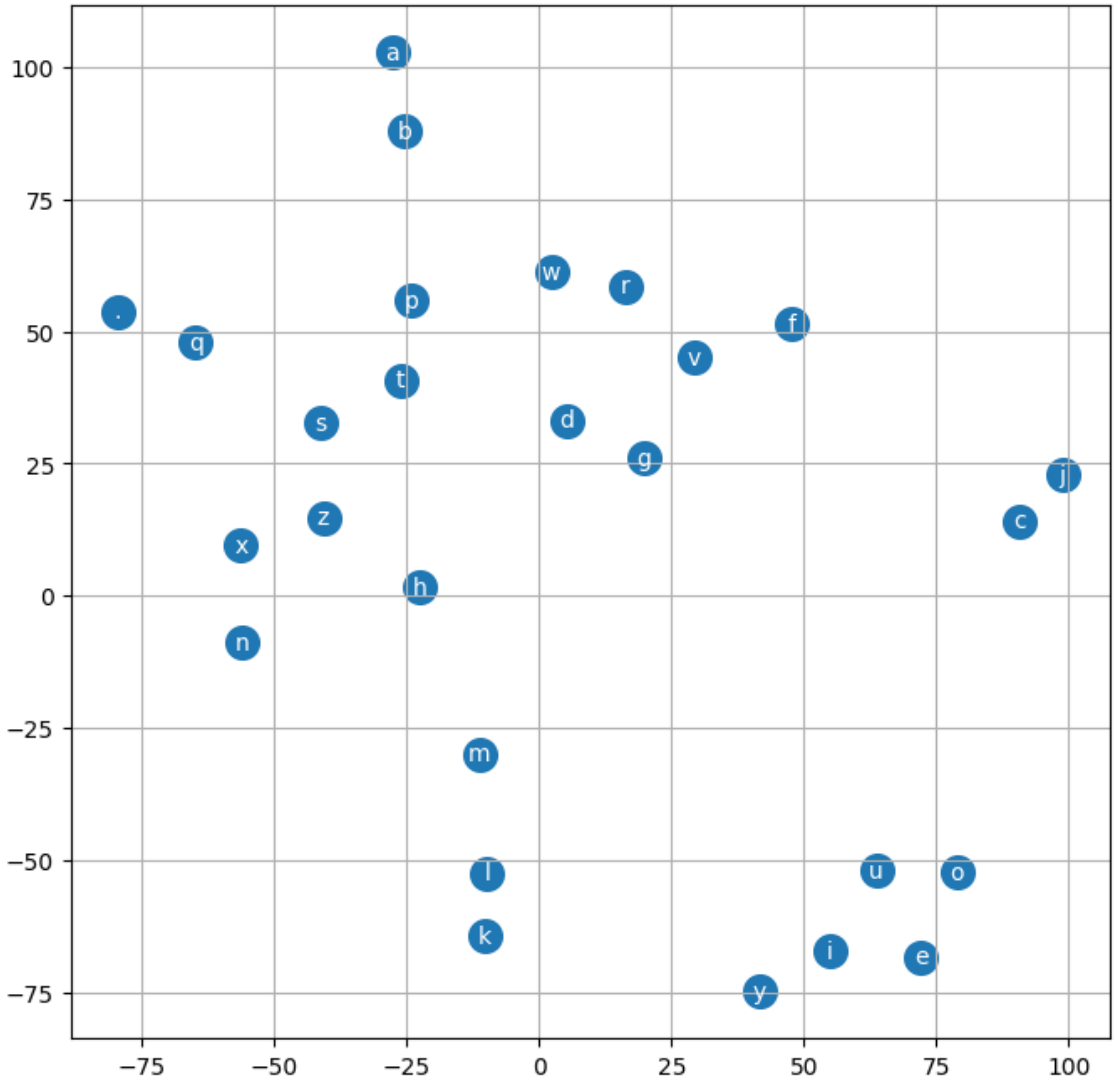
```
>>> print([vocab[token] for token in ['this', 'is',
...     'an', 'example']])
[11, 7, 35, 457]
```

- make a dictionary of tokens
- normal supervised learning classification problem
- given words 11, 7, and 35, learn to predict that word 457 is next

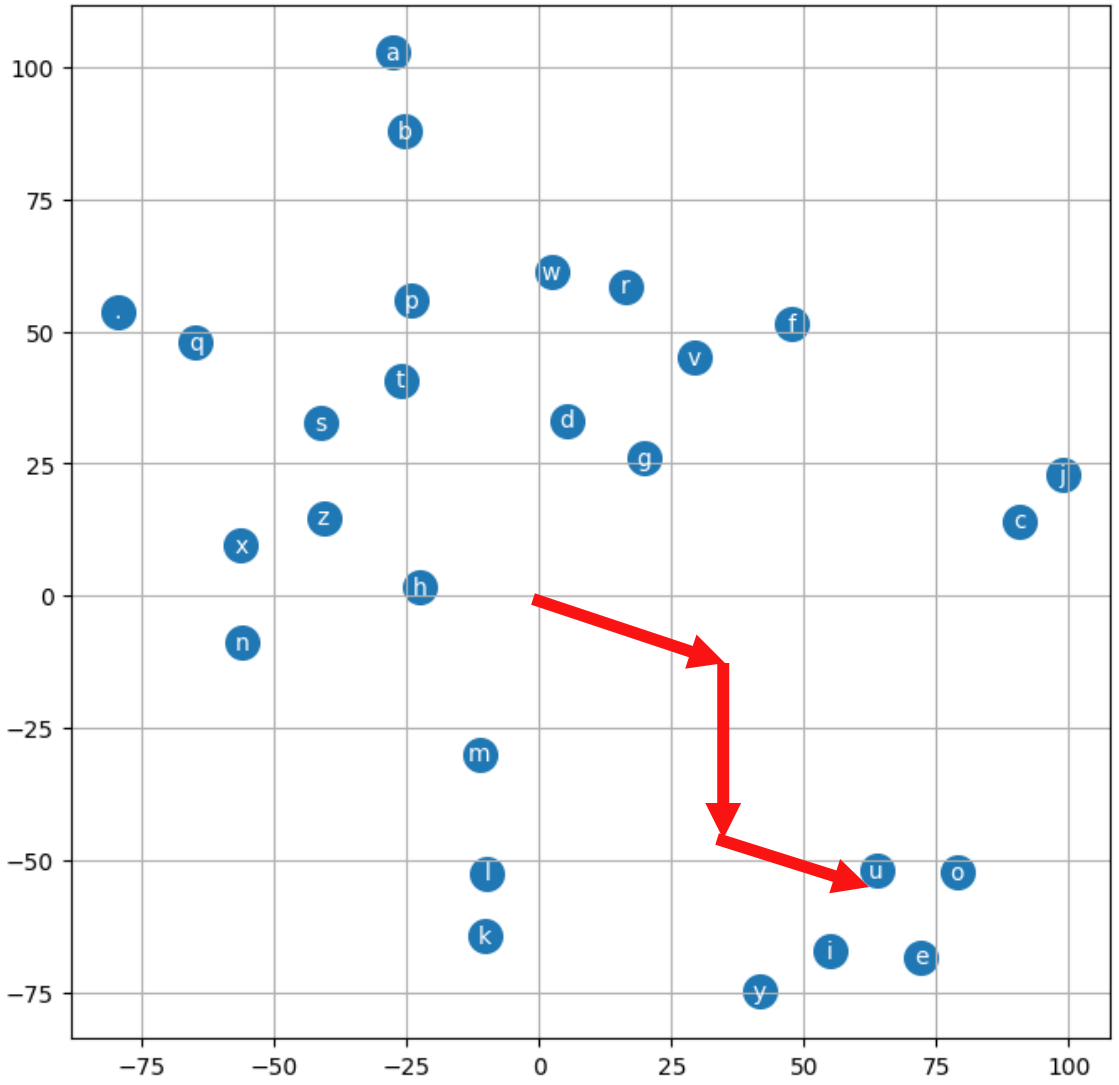
Why Embedding?



- Store tokens in high-dimension, continuous vector space where direction has some semantic meaning
- Vector for the output token starts at origin
- all of the surrounding context tokens calculate displacements that we sum to push output vector in certain direction
- make final choice based on closest tokens



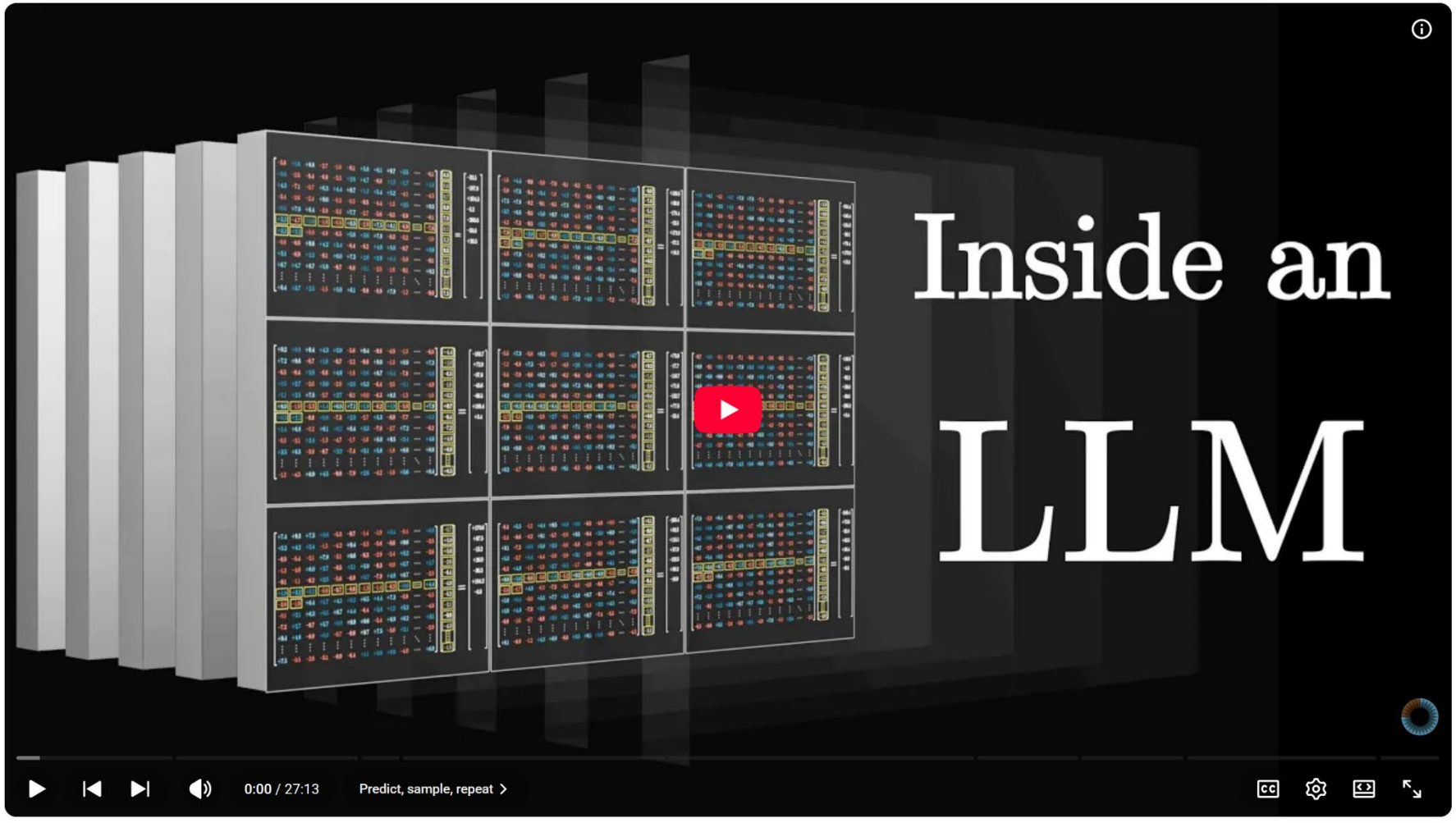
- N-gram with embedding
- Still using names data
- using tSNE to draw embedding matrix
- notice cluster of vowels in corner



- Now imagine our output starts as a vector at origin
- We calculate multiple shifts from context
- Choose output based on nearby tokens

https://www.youtube.com/watch?v=wjZofJX0v4M&list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi&index=8

Embedding from 12:30 to 18:30



```
1 !pip install gensim
```

```
Collecting gensim
  Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.w
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (fro
Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 27.9/27.9 MB 27.3 MB/s eta 0:00:00
Installing collected packages: gensim
Successfully installed gensim-4.4.0
```

```
1 import gensim.downloader
```

```
1 model = gensim.downloader.load("glove-wiki-gigaword-50")   # 66MB data file
```

```
[==================================================] 100.0% 66.0/66.0MB downloaded
```

```
1 model['man']
```

```
array([-0.094386,  0.43007 , -0.17224 , -0.45529 ,  1.6447  ,  0.40335 ,
       -0.37263 ,  0.25071 , -0.10588 ,  0.10778 , -0.10848 ,  0.15181 ,
       -0.65396 ,  0.55054 ,  0.59591 , -0.46278 ,  0.11847 ,  0.64448 ,
       -0.70948 ,  0.23947 , -0.82905 ,  1.272   ,  0.033021,  0.2935  ,
        0.3911  , -2.8094  , -0.70745 ,  0.4106  ,  0.3894  , -0.2913  ,
        2.6124  , -0.34576 , -0.16832 ,  0.25154 ,  0.31216 ,  0.31639 ,
        0.12539 , -0.012646,  0.22297 , -0.56585 , -0.086264,  0.62549 ,
       -0.0576  ,  0.29375 ,  0.66005 , -0.53115 , -0.48233 , -0.97925 ,
        0.53135 , -0.11725 ], dtype=float32)
```

```
1 print(model.most_similar(positive=['car', 'minivan'], topn=5))
```

```
[('truck', 0.9100273251533508), ('suv', 0.904007613658905), ('jeep', 0.8619828820228577), ('vehicle', 0.8431736826896667), ('cars', 0.8421834111213684)]
```

```
d = model['hitler'] + model['italy'] - model['germany']
model.similar_by_vector(d, topn=3)
```

```
[('mussolini', 0.8470991849899292),
 ('hitler', 0.7255331873893738),
 ('fascist', 0.6964704990386963)]
```

```
d = model['sushi'] + model['germany'] - model['japan']
model.similar_by_vector(d, topn=3)
```

```
[('gourmet', 0.6915086507797241),
 ('fries', 0.6719846725463867),
 ('sausages', 0.6515130400657654)]
```

```
d = model['sushi'] + model['america'] - model['japan']
model.similar_by_vector(d, topn=3)
```

```
[('gourmet', 0.7495439052581787),
 ('chefs', 0.6717949509620667),
 ('tasting', 0.6674089431762695)]
```

```
for v in ['zero','one', 'two','three','four']:
  print(v, np.dot(model['cats'] - model['cat'], model[v]))
```

```
zero -1.1029267
one -2.399511
two 0.7914256
three 1.2681851
four 1.4922795
```

TRANSFORMERS

Transformers

More examples of these later. Goal is to connect different representations:

- encoders – transform input (text, image, speech) into generic representation
- decoder – predict next token of text/sound/image
- encoder-decoder – translate different languages or inputs

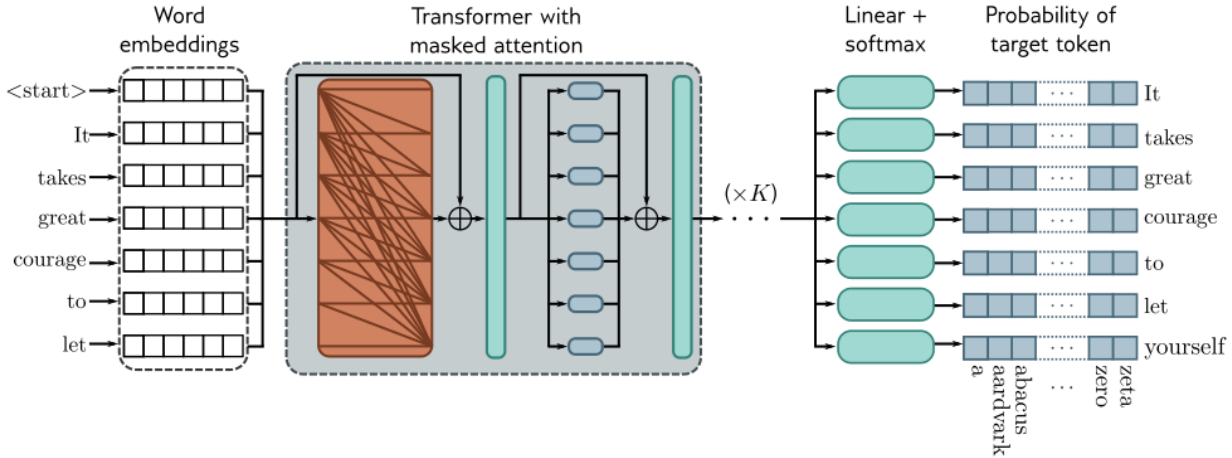
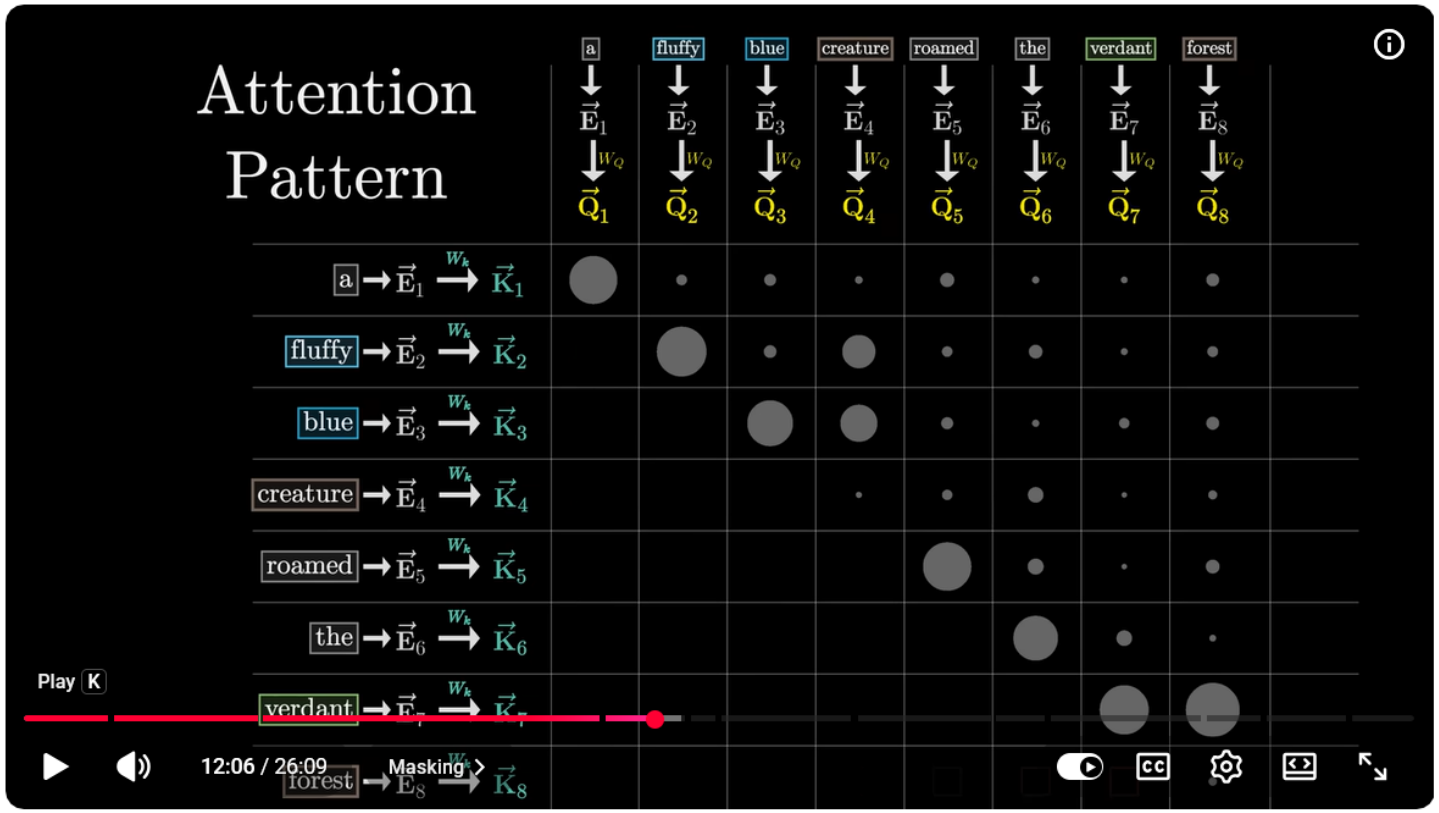


Figure 12.12 Training GPT3-type decoder network. The tokens are mapped to word embeddings with a special <start> token at the beginning of the sequence. The embeddings are passed through a series of transformer layers that use masked self-attention. Here, each position in the sentence can only attend to its own embedding and those of tokens earlier in the sequence (orange connections). The goal at each position is to maximize the probability of the following ground truth token in the sequence. In other words, at position one, we want to maximize the probability of the token It; at position two, we want to maximize the probability of the token takes; and so on. Masked self-attention ensures the system cannot cheat by looking at subsequent inputs. The autoregressive task has the advantage of making efficient use of the data since every word contributes a term to the loss function. However, it only exploits the left context of each word.

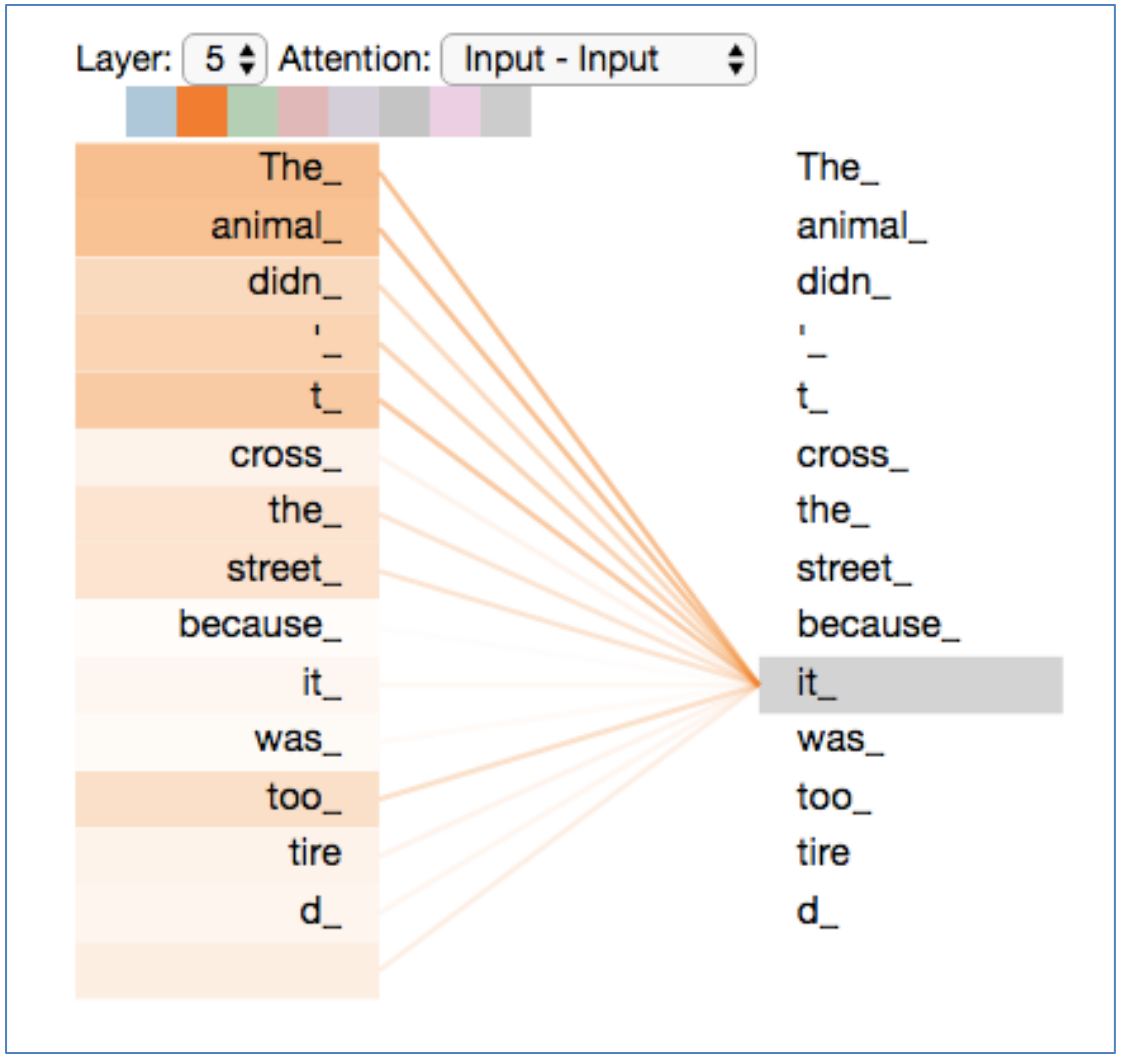
https://www.youtube.com/watch?v=eMlx5fFNoYc&list=PLZHQ
ObOWTQDNU6R1_67000Dx_ZCJB-3pi&index=7



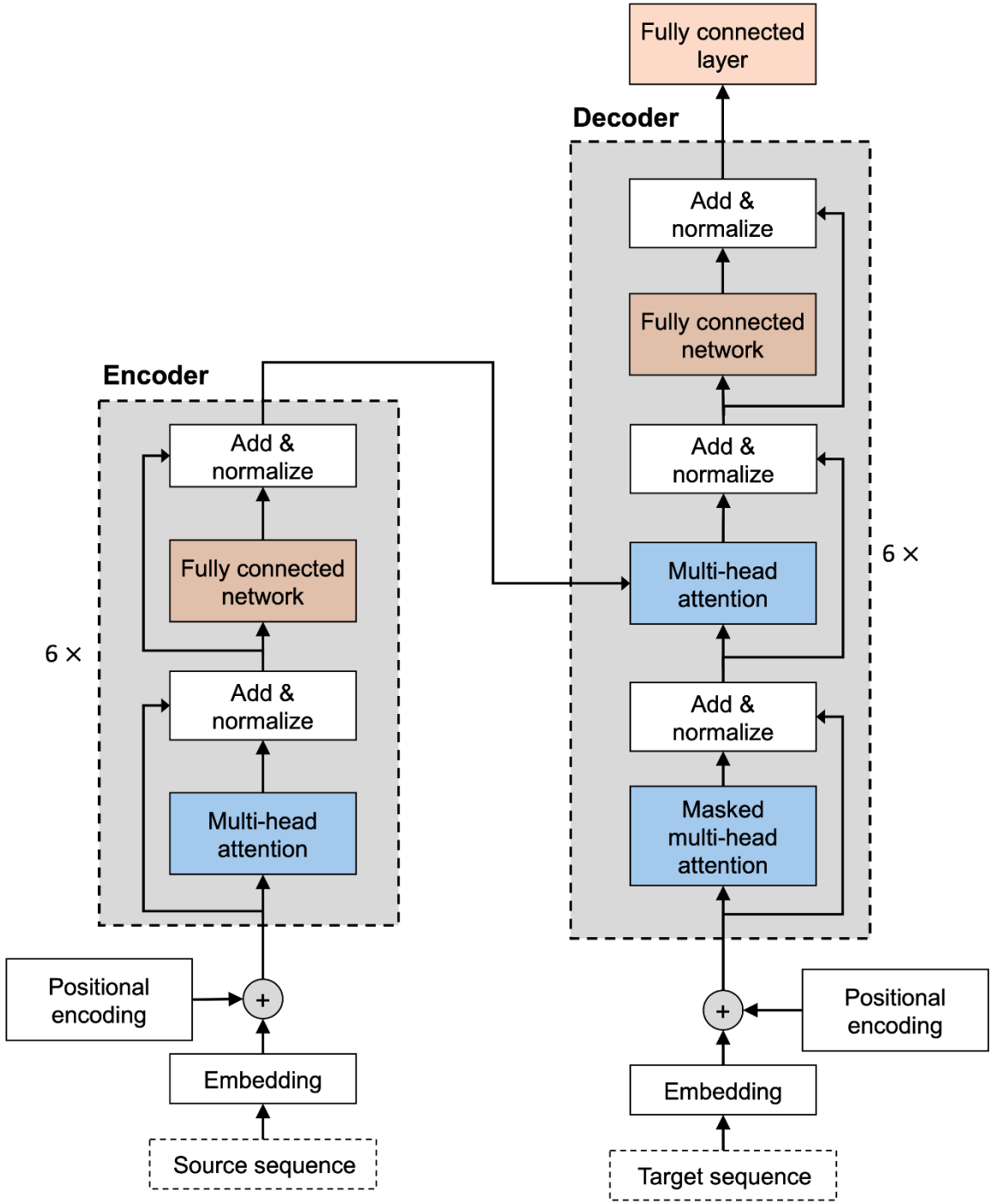
Attention in transformers, step-by-step | Deep Learning Chapter 6

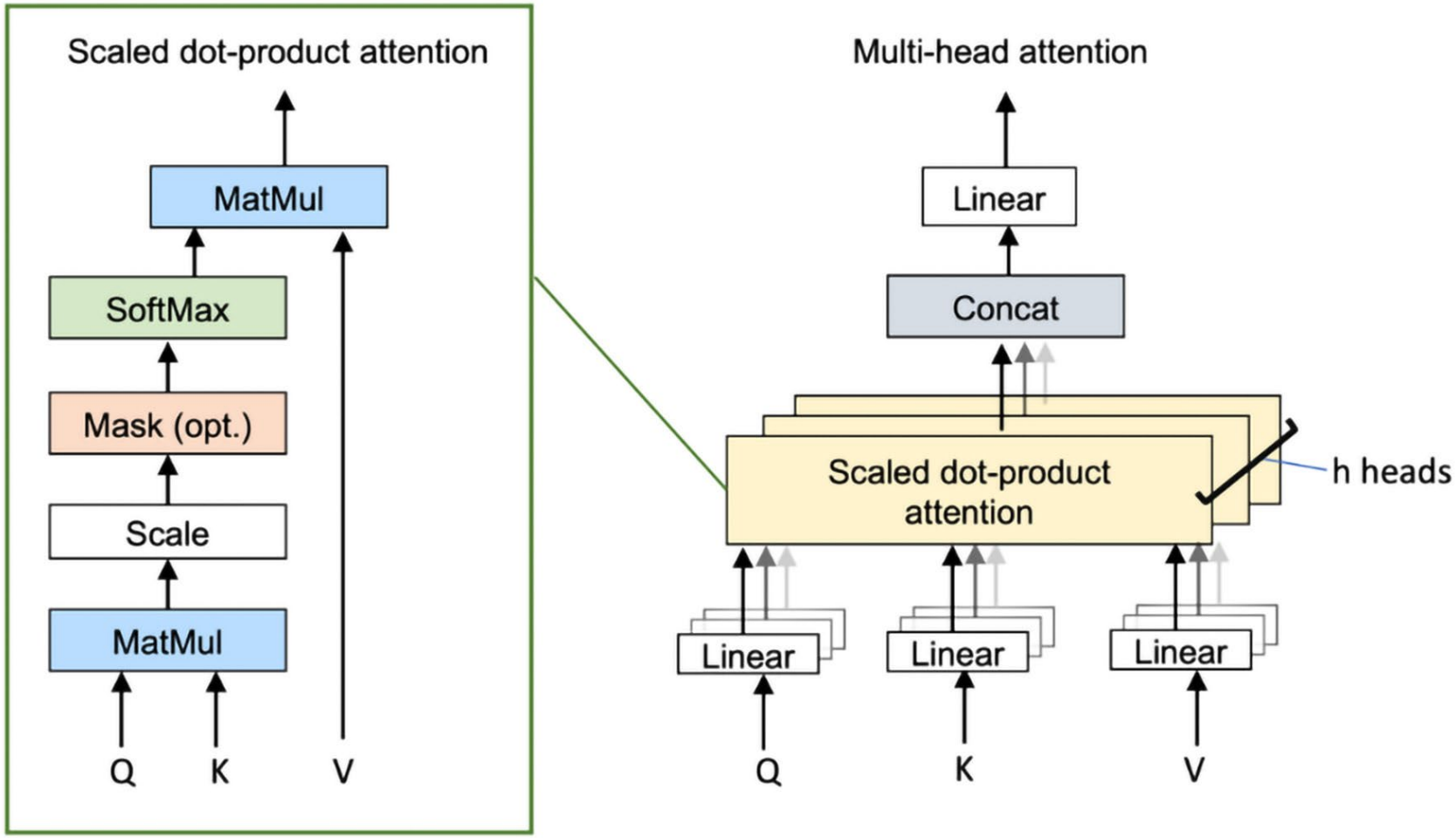
3Blue1Brown

83K Share Ask Save



http://jalammar.github.io/illustrated-transformer/





- query Q = input token
- key K, tokens are related if the dot product $K \cdot Q$ is large
- value V: output, which direction to shift output if K and Q match

- https://karpathy.ai/zero-to-hero.html
- https://colab.research.google.com/drive/1JMLa53HDuA-i7ZBmqV7ZnA3c_fvtXnx-?usp=sharing#scrollTo=ZcvKeBXoZFOY
- https://github.com/karpathy/nanoGPT/tree/master

```python
class BigramLanguageModel(nn.Module):

    def __init__(self):
        super().__init__()
        # each token directly reads off the logits for the next token from a lookup table
        self.token_embedding_table = nn.Embedding(vocab_size, n_embd)
        self.position_embedding_table = nn.Embedding(block_size, n_embd)
        self.blocks = nn.Sequential(*[Block(n_embd, n_head=n_head) for _ in range(n_layer)])
        self.ln_f = nn.LayerNorm(n_embd) # final layer norm
        self.lm_head = nn.Linear(n_embd, vocab_size)

    def forward(self, idx, targets=None):
        B, T = idx.shape

        # idx and targets are both (B,T) tensor of integers
        tok_emb = self.token_embedding_table(idx) # (B,T,C)
        pos_emb = self.position_embedding_table(torch.arange(T, device=device)) # (T,C)
        x = tok_emb + pos_emb # (B,T,C)
        x = self.blocks(x) # (B,T,C)
        x = self.ln_f(x) # (B,T,C)
        logits = self.lm_head(x) # (B,T,vocab_size)

        if targets is None:
            loss = None
        else:
            B, T, C = logits.shape
            logits = logits.view(B*T, C)
```

GPTs for LLMs

- The attention mechanism in transforms gives a great solution to the problems of LLMs
  - learns connections of words in different contexts
  - learns how context shifts output
- Training a model like this on a huge block of input text will learn to reliably generate similar text, but won't be a helpful AI assistant on its own
- The next training step is **REINFORCEMENT LEARNING**
  - have humans label helpful responses
  - turn this data into reward model
  - use reinforcement learning to produce more helpful responses